Modul 7

Integritätskontrolle und aktives Verhalten

Josef Altmann



Der vorliegende Foliensatz basiert auf:

Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Konzepte und Sprachen*, MITP, 6. Auflage (2018)
Kemper, A., Eickler, A.: *Datenbanksysteme: Eine Einführung*, Oldenbourg, 10. Auflage (2015)
Loney, K: *Oracle Database 11g – Die umfassende Referenz*, Hanser Verlag (2009)
PL/SQL Language Reference, docs.oracle.com/cd/B28359_01/appdev.111/b28370.pdf

Überblick

- Einführung
- Deklarative Integritätsbedingungen
- Prozedurale Integritätsbedingungen

© 2026

Integritätsbedingungen - Begriffsklärung

Integrität versus Konsistenz

- **Integrität**

  korrekte, vollständige Abbildung der (realen) Miniwelt
  - Integrität kann verletzt sein
  - obwohl Konsistenz der DB gewahrt bleibt
  - DBS kann nur die Konsistenz der Daten sichern!

- **Konsistenz**

  (korrekter) DB interner Zusammenhang von
  - Speicherungsstrukturen
  - Zugriffspfaden und
  - sonstigen Verwaltungsinformationen

- **Constraints**

  Sprachkonzepte für die Sicherstellung der Konsistenz
  - Wertebereiche
  - Primärschlüssel
  - Fremdschlüssel
  - Check-Klauseln

© 2026

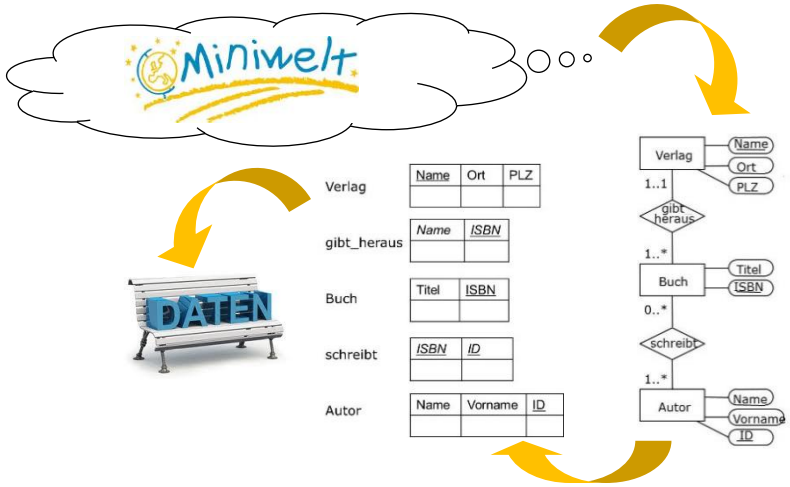
Integritätsbedingungen - Begriffsklärung

Integrität versus Konsistenz

- Trotzdem spricht man bei Datenbanken von **Integritätssicherung**
  - also Referentielle Integrität anstatt Referentielle Konsistenz
  - Constraints (Integritätsbedingungen) spezifizieren
    - akzeptable DB-Zustände
    - nicht aktuelle Zustände der Miniwelt
  - Änderungen werden nur zurückgewiesen, wenn sie
    - entsprechend der Integritätsbedingungen als falsch erkannt werden

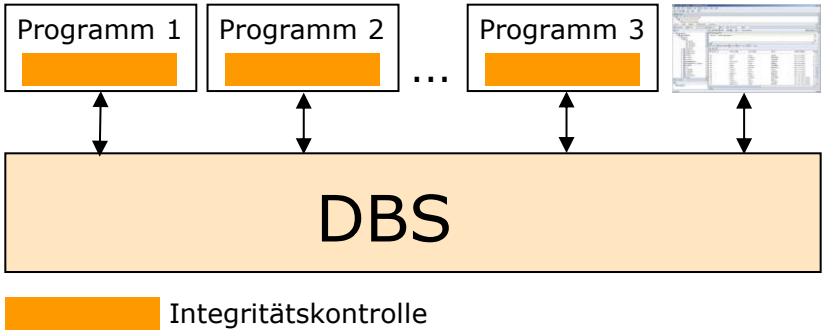
Integritätsbedingungen
Ziele

- Änderungen müssen Constraints entsprechen
  - "falsche" Änderungen zurückweisen
- Hohe Übereinstimmung von
  - DB-Inhalt und
  - Miniwelt (Datenqualität)
- Voraussetzung
  - Integritätsbedingungen der Miniwelt explizit bekannt
  - ermöglicht automatische Integritätskontrolle



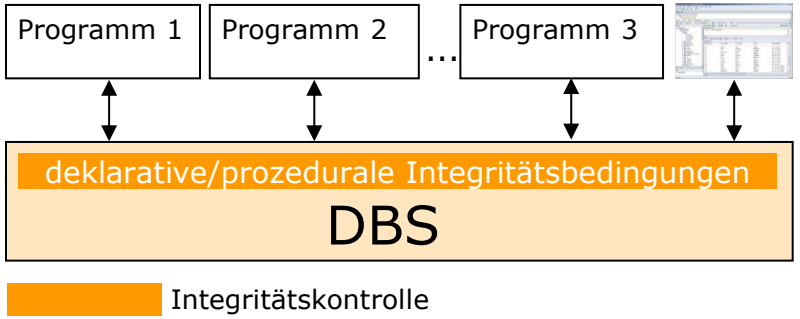
Integritätsbedingungen
Kontrolle

- Überwachung von Integritätsbedingungen durch das Anwendungsprogramm



  - flexibel und effizient
  - geringere Last auf DBMS
  - Code-Redundanzen
  - Code-Inkonsistenzen
    - verschiedene Prüfungen

- DBS-basierte Integritätskontrolle



  - sicherer, verlässlicher und schneller
  - einfachere Anwendungserstellung
  - Unterstützung von interaktiven sowie programmierten DB-Änderungen
  - leichtere Änderbarkeit von Integritätsbedingungen
  - zentrale Kontrolle durch das DBMS
  - höhere Last auf DBMS (DBMS als Flaschenhals)
  - Vendor Lock-in-Effekt (herstellerspezifische SQL-Sprachvariationen)

Integritätsbedingungen

Klassifikation - Performanz und Testbarkeit

Kriterium	Klassifikation
Granularität von Daten bzw. Reichweite der Änderung	▪ ein Attribut ▪ mehrere Attribute eines Tupels ▪ mehrere Tupel (selbe Relation) ▪ mehrere Tupel (vers. Relationen)
Zeitpunkt der Überprüfung	▪ Sofort (bei Änderungsoperation) ▪ Verzögert (am Transaktionsende)
Art der Überprüfbarkeit	▪ Statisch ▪ Dynamisch ▪ Transitional (Übergangsprüfung) ▪ Temporal
Art der Reaktion	▪ Zurückweisung (reject) ▪ Korrektur (repair)
Ausdrucksfähigkeit der Sprache	▪ Deklarativ ▪ Prozedural/Operational

© 2026

Integritätsbedingungen

Klassifikation - Performanz und Testbarkeit

Kriterium	Klassifikation
Granularität von Daten bzw. Reichweite der Änderung	▪ ein Attribut ▪ mehrere Attribute eines Tupels ▪ mehrere Tupel (selbe Relation) ▪ mehrere Tupel (vers. Relationen)
Zeitpunkt der Überprüfung	▪ Sofort (bei Änderungsoperation) ▪ Verzögert (am Transaktionsende)
Art der Überprüfbarkeit	▪ Statisch ▪ Dynamisch ▪ Transitional (Übergangsprüfung) ▪ Temporal
Art der Reaktion	▪ Zurückweisung (reject) ▪ Korrektur (repair)
Ausdrucksfähigkeit der Sprache	▪ Deklarativ ▪ Prozedural/Operational

© 2026

Integritätsbedingungen

Klassifikation - Performanz und Testbarkeit

Kriterium	Klassifikation
Granularität von Daten bzw. Reichweite der Änderung	▪ ein Attribut ▪ mehrere Attribute eines Tupels ▪ mehrere Tupel (selbe Relation) ▪ mehrere Tupel (vers. Relationen)
Zeitpunkt der Überprüfung	▪ Sofort (bei Änderungsoperation) ▪ Verzögert (am Transaktionsende)
Art der Überprüfbarkeit	▪ Statisch ▪ Dynamisch ▪ Transitional (Übergangsprüfung) ▪ Temporal
Art der Reaktion	▪ Zurückweisung (reject) ▪ Korrektur (repair)
Ausdrucksfähigkeit der Sprache	▪ Deklarativ ▪ Prozedural/Operational

© 2026

Integritätsbedingungen
Art der Überprüfbarkeit

- **Statische Integritätsbedingungen**
  Beziehen sich auf den Datenbankzustand → SQL DDL
  - Kundennummer muss eindeutig sein
  - Summe der Gehälter ≤ Gehaltsbudget der Abteilung

- **Dynamische Integritätsbedingungen**
  Beziehen sich auf Zustandsübergänge → Prozedurales SQL
  - Transitional: Übergang von altem zu neuem DB-Zustand wird eingeschränkt
    - Familienstand „ledig" ↛ „geschieden"
    - Preiserhöhung max. 10%
  - Temporal: Änderungen in bestimmtem zeitlichen Fenster werden eingeschränkt
    - Gehalt darf innerhalb von drei Jahren nicht um mehr als 25 % steigen

© 2026

Integritätsbedingungen

Klassifikation - Performanz und Testbarkeit

Kriterium	Klassifikation
Granularität von Daten bzw. Reichweite der Änderung	▪ ein Attribut ▪ mehrere Attribute eines Tupels ▪ mehrere Tupel (selbe Relation) ▪ mehrere Tupel (vers. Relationen)
Zeitpunkt der Überprüfung	▪ Sofort (bei Änderungsoperation) ▪ Verzögert (am Transaktionsende)
Art der Überprüfbarkeit	▪ Statisch ▪ Dynamisch ▪ Transitional (Übergangsprüfung) ▪ Temporal
Art der Reaktion	▪ Zurückweisung (reject) ▪ Korrektur (repair)
Ausdrucksfähigkeit der Sprache	▪ Deklarativ ▪ Prozedural/Operational

© 2026

Integritätsbedingungen

Klassifikation - Performanz und Testbarkeit

Kriterium	Klassifikation
Granularität von Daten bzw. Reichweite der Änderung	▪ ein Attribut ▪ mehrere Attribute eines Tupels ▪ mehrere Tupel (selbe Relation) ▪ mehrere Tupel (vers. Relationen)
Zeitpunkt der Überprüfung	▪ Sofort (bei Änderungsoperation) ▪ Verzögert (am Transaktionsende)
Art der Überprüfbarkeit	▪ Statisch ▪ Dynamisch ▪ Transitional (Übergangsprüfung) ▪ Temporal
Art der Reaktion	▪ Zurückweisung (reject) ▪ Korrektur (repair)
Ausdrucksfähigkeit der Sprache	▪ Deklarativ ▪ Prozedural/Operational

© 2026

Integritätsbedingungen

Ausdrucksfähigkeit der Sprache

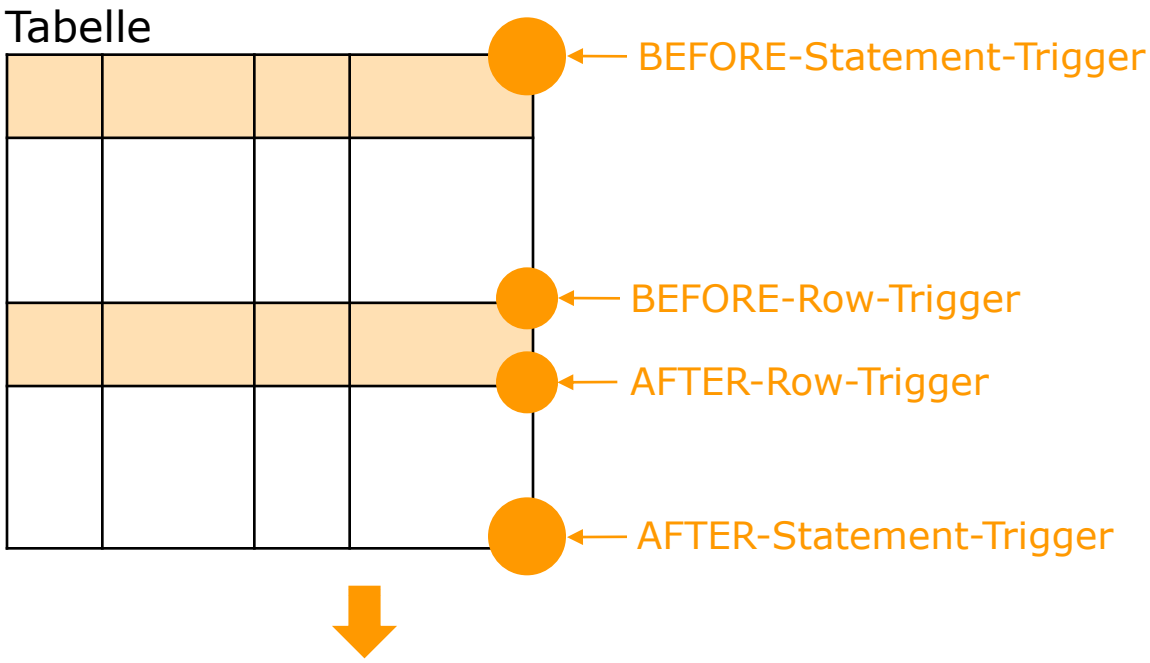
Deklarative IB

- PRIMARY KEY
- UNIQUE
- NOT NULL/NULL
- FOREIGN KEY/ REFERENCES
- CHECK
- ASSERTION
- (DEFAULT)

geringe Reichweite, statisch (einfach), sofort/verzögert, zurückweisend

Prozedurale IB

- TRIGGER



größere Reichweite, dynamisch (komplex), sofort, Korrekturmaßnahmen möglich

Überblick

- Einführung
- Deklarative Integritätsbedingungen
- Prozedurale Integritätsbedingungen

Deklarative Integritätsbedingungen

- Als Attribut- oder als Tabellen-Integritätsbedingung spezifiziert
- Name kann vergeben werden, über den später auf die Integritätsbedingung Bezug genommen werden kann

```
[CONSTRAINT <name>]   PRIMARY KEY [(<Attribute>)]
[CONSTRAINT <name>]   UNIQUE [(<Attribute>)]
[CONSTRAINT <name>]   [NOT] NULL
[CONSTRAINT <name>]   [FOREIGN KEY [(<Attribute>)]
                      REFERENCES <Tabelle> [(<Attribute>)]
                      ON DELETE CASCADE | SET NULL
[CONSTRAINT <name>]   CHECK (<Ausdruck>)
```

Deklarative Integritätsbedingungen

Referentielle Integrität- FOREIGN KEY

```
CREATE TABLE dept (
 deptno NUMBER(8),
 ort VARCHAR2(50) NOT NULL,
 CONSTRAINT dept_pk
         PRIMARY KEY (deptno));
```

dept

deptno	ort
1	Linz
2	Sbg
3	Wien
4	Linz

```
DELETE FROM dept
WHERE deptno=3;

UPDATE dept
SET deptno=5
WHERE deptno=1;
```

```
ORA-02292:
Verstoß gegen Constraint (EMP_FK).
Untergeordneter Datensatz gefunden.
```

```
CREATE TABLE emp (
 empno NUMBER(8),
 in_dept NUMBER(8),
 CONSTRAINT emp_pk
         PRIMARY KEY (empno),
 CONSTRAINT emp_fk
         FOREIGN KEY (in_dept)
         REFERENCES dept);
```

emp

empno	in_dept
100	1
200	2
300	2
400	3
500	4

```
INSERT INTO emp
          (empno, in_dept)
          VALUES (600, 5);

UPDATE emp
SET in_dept=5
WHERE empno=300;
```

```
ORA-02291:
Verstoß gegen Constraint (EMP_FK).
Übergeordn. Schlüssel nicht gefunden.
```

Deklarative Integritätsbedingungen

Referentielle Aktionen – ON DELETE und ON UPDATE

- Spezifikation der **referentiellen Aktionen** pro Fremdschlüssel
  - Ist NULL verboten?
    - NOT NULL
  - Was passiert beim Löschen des referenzierten Werts?
    - ON DELETE
      {CASCADE|RESTRICT|SET NULL|SET DEFAULT|NO ACTION}
  - Was passiert beim Ändern des referenzierten Werts?
    - ON UPDATE (nicht in Oracle)
      {CASCADE|RESTRICT|SET NULL|SET DEFAULT|NO ACTION}
  - NO ACTION ist Standardeinstellung in Oracle
    kann aber nicht angegeben werden

Deklarative Integritätsbedingungen

Referentielle Aktionen – `ON DELETE` und `ON UPDATE`

- **`RESTRICT`**
  Operation wird nur ausgeführt, wenn keine zugehörigen Sätze (FS-Werte) vorhanden sind
- **`CASCADE`**
  Operation propagiert zu allen zugehörigen Sätzen
- **`SET NULL`**
  FS wird in zugehörigen Sätzen auf `NULL` gesetzt
- **`SET DEFAULT`**
  FS wird in den zugehörigen Sätzen auf einen benutzerdefinierten Default-Wert gesetzt
- **`NO ACTION`**
  - Für die spezifizierte Referenz wird keine referentielle Aktion ausgeführt.
  - Durch eine DB-Operation können jedoch mehrere Referenzen (mit unterschiedlichen Optionen) betroffen sein;
  - Am Ende aller zugehörigen referentiellen Aktionen wird die Einhaltung der referentiellen Integrität geprüft

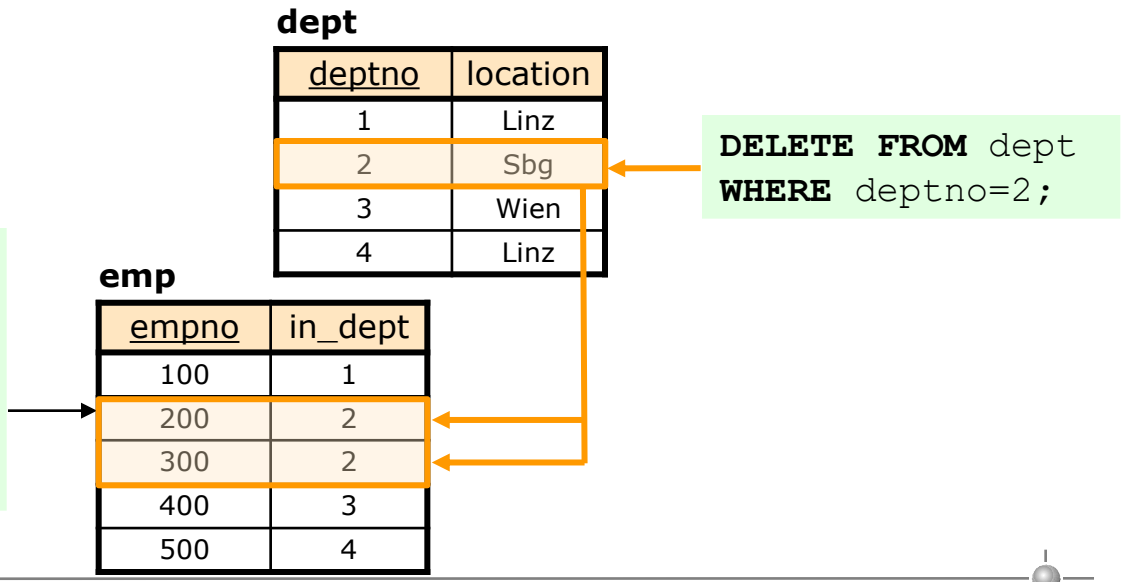
© 2026

Deklarative Integritätsbedingungen

Referentielle Aktionen – ON DELETE

- ON DELETE...
  - CASCADE
  - SET NULL
- Abhängige Datensätze in Kindtabelle werden **vor** dem Datensatz in der Vatertabelle gelöscht bzw. auf NULL gesetzt
- Bei **mehrstufigen Abhängigkeiten** muss auf **allen Ebenen** die ON DELETE-Option existieren

```
CREATE TABLE emp (
 ....,
 CONSTRAINT emp_fk
         FOREIGN KEY (in_dept)
         REFERENCES dept
         ON DELETE CASCADE);
```



Deklarative Integritätsbedingungen

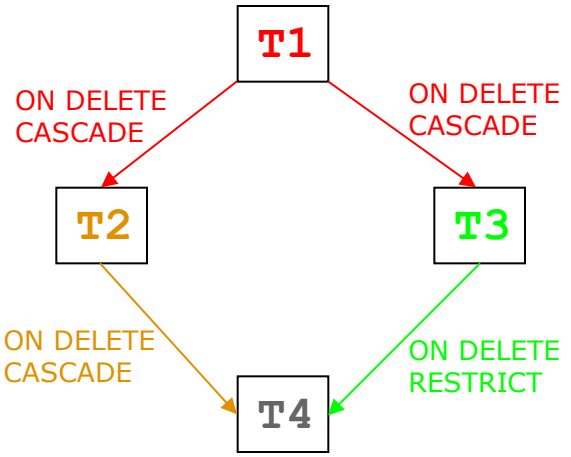
Referentielle Aktionen – Abarbeitung von FOREIGN KEY-Klauseln

```
CREATE TABLE T1 (... PRIMARY KEY K1)

CREATE TABLE T2 (... PRIMARY KEY K2
  FOREIGN KEY (K2) REFERENCES T1(K1)
  ON DELETE CASCADE)

CREATE TABLE T3 (... PRIMARY KEY K3
  FOREIGN KEY (K3) REFERENCES T1(K1)
  ON DELETE CASCADE)

CREATE TABLE T4 (... PRIMARY KEY (K4,K5)
  FOREIGN KEY (K4) REFERENCES T3(K3)
  ON DELETE RESTRICT,
  FOREIGN KEY (K5) REFERENCES T2(K2)
  ON DELETE CASCADE)
```



DELETE in T1:
Aktionen von T2 vor T4 vor T3 ✓
Aktionen von T3 vor T4 vor T2 ↯

⇨ Es können reihenfolgenabhängige Ergebnisse auftreten!

Deklarative Integritätsbedingungen

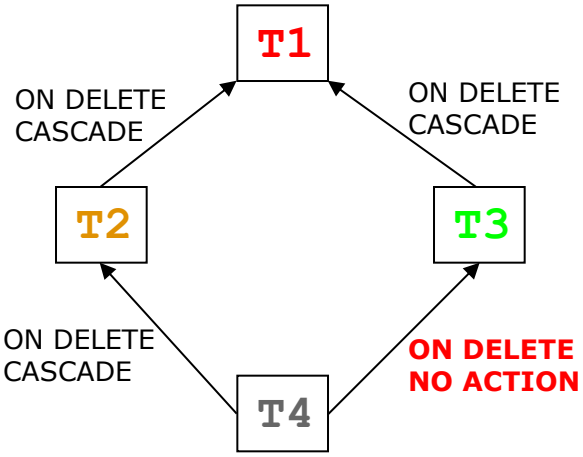
Referentielle Aktionen – Abarbeitung von FOREIGN KEY-Klauseln

```
CREATE TABLE T1 (... PRIMARY KEY K1)

CREATE TABLE T2 (... PRIMARY KEY K2
  FOREIGN KEY (K2) REFERENCES T1(K1)
  ON DELETE CASCADE)

CREATE TABLE T3 (... PRIMARY KEY K3
  FOREIGN KEY (K3) REFERENCES T1(K1)
  ON DELETE CASCADE)

CREATE TABLE T4 (... PRIMARY KEY (K4,K5)
  FOREIGN KEY (K4) REFERENCES T3(K3)
  ON DELETE RESTRICT, NO ACTION,
  FOREIGN KEY (K5) REFERENCES T2(K2)
  ON DELETE CASCADE)
```



Bei der NO ACTION-Option wird der explizite Test der referenzierenden Relation ans Ende der Operation verschoben.
Eine Verletzung der referentiellen Beziehung führt zum Rücksetzen
⇨ **Schema ist immer sicher**

Deklarative Integritätsbedingungen

Referentielle Aktionen - Systemunterstützung

SQL-1999 **FOREIGN KEY** **ON DELETE**	Oracle	DB2	Postgres
NO ACTION	(✓)	✓	✓
RESTRICT	–	✓	✓
CASCADE	✓	✓	✓
SET NULL	✓	✓	✓
SET DEFAULT	–	–	✓
ON UPDATE			
NO ACTION	(✓)	✓	✓
RESTRICT	–	✓	✓
CASCADE	–	–	✓
SET NULL	–	–	✓
SET DEFAULT	–	–	✓

Deklarative Integritätsbedingungen

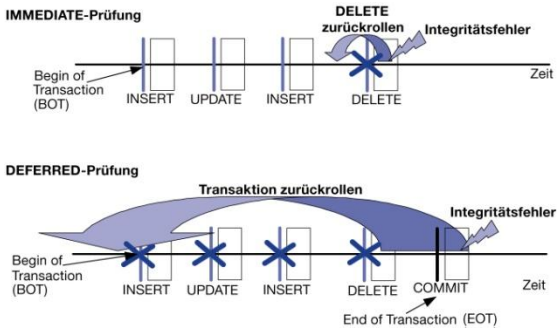
Überprüfungsmodi 1/3

Notation

```
[CONSTRAINT <Name>] <Constraint-Definition>
[INITIALLY (IMMEDIATE | DEFERRED)]
[[NOT] DEFERRABLE]; *
```

* Standardeinstellung

- Constraint (IB) hat
  - eindeutigen Namen
    - explizit definiert mit `CONSTRAINT`-Klausel oder
    - implizit generiert vom DBS
  - Überprüfungsmodus
    - bestimmt innerhalb einer laufenden Transaktion den Zeitpunkt, an dem der Constraint überprüft wird
      - » `INITIALLY IMMEDIATE` überprüft Constraint unmittelbar nach jeder SQL-Anweisung
      - » `INITIALLY DEFERRED` verzögert die Überprüfung an das Transaktionsende



  - Verzögerungsmodus
    - Modus `DEFERRED` können nur verzögerbare Constraints annehmen
    - Verzögerbarkeit wird
      - » implizit durch Klausel `INITIALLY DEFERRED` oder
      - » explizit durch Klausel `DEFERRABLE` gesetzt

Deklarative Integritätsbedingungen

Überprüfungsmodi 2/3

- Dadurch können bei kaskadierenden Update-Operationen z.B. in Vater-Kind-Tabellen diese Operationen in beliebiger Reihenfolge ausgeführt werden
- Vor Transaktionende können mit dem Befehl
  - `SET CONSTRAINTS ALL IMMEDIATE;`
  - `SET CONSTRAINT <Constraint> IMMEDIATE;`

  Constraint-Verletzungen geprüft und entsprechend behandelt werden
- Über `INITIALLY IMMEDIATE DEFERRABLE` kann zum Definitionszeitpunkt festgelegt werden, dass grundsätzlich alle Constraints sofort geprüft werden, zur Laufzeit kann jedoch über `SET CONSTRAINTS` auf `DEFERRED` umgeschaltet werden

Deklarative Integritätsbedingungen

Überprüfungsmodi 3/3

Initialeinstellung \ Aufschiebbarkeit	NOT DEFERRABLE	DEFERRABLE
INITIALLY IMMEDIATE	Standard	SET CONSTRAINT...DEFERRED; SET CONSTRAINT...IMMEDIATE;
INITIALLY DEFERRED	nicht möglich	SET CONSTRAINT...IMMEDIATE;

```
CREATE TABLE emp (
    empno NUMBER(8),
    in_dept NUMBER(8),
    CONSTRAINT emp_pk PRIMARY KEY (empno),
    CONSTRAINT emp_fk FOREIGN KEY (in_dept) REFERENCES dept
       INITIALLY IMMEDIATE DEFERRABLE);
...
  SET CONSTRAINT emp_fk DEFERRED;
  INSERT INTO emp VALUES (700, 10);
  INSERT INTO dept VALUES (10, 'Sbg');
  SET CONSTRAINT emp_fk IMMEDIATE;
```

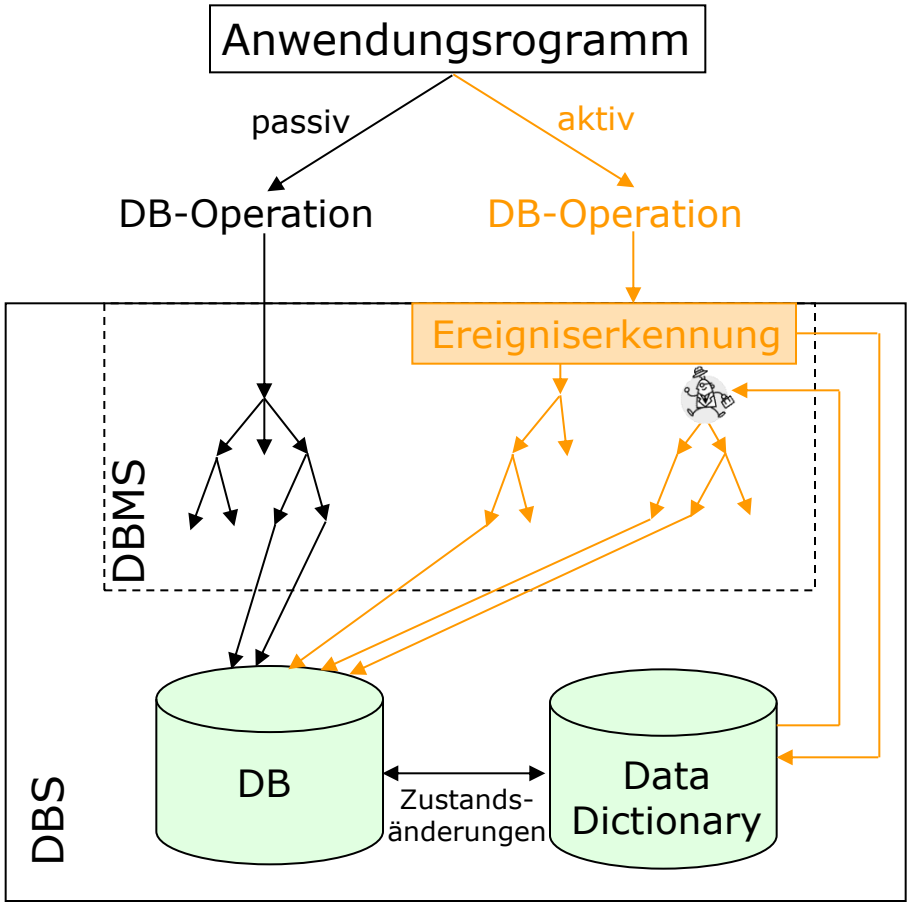
Worin liegt der Unterschied zwischen IMMEDIATE/DEFERRED und ENABLE/DISABLE von Integritätsbedingungen?

Überblick

- Einführung
- Deklarative Integritätsbedingungen
- Prozedurale Integritätsbedingungen
  - Aktives Verhalten
  - Trigger-Komponenten
  - Ausführungsmodell

Prozedurale Integritätsbedingungen

Aktives Verhalten 1/2



- Ausführung von DB-Operationen
  - Wirkung der DB-Operation im DBS festgelegt
  - Expliziter Aufruf der DB-Operation erforderlich
  - DBS ist letztendlich "passiv"!
- Wo beginnt aktives DBS-Verhalten?
  - Erkennung von Ereignissen oder Zustandsänderungen
  - "selbständige" Durchführung von Operationen

Prozedurale Integritätsbedingungen

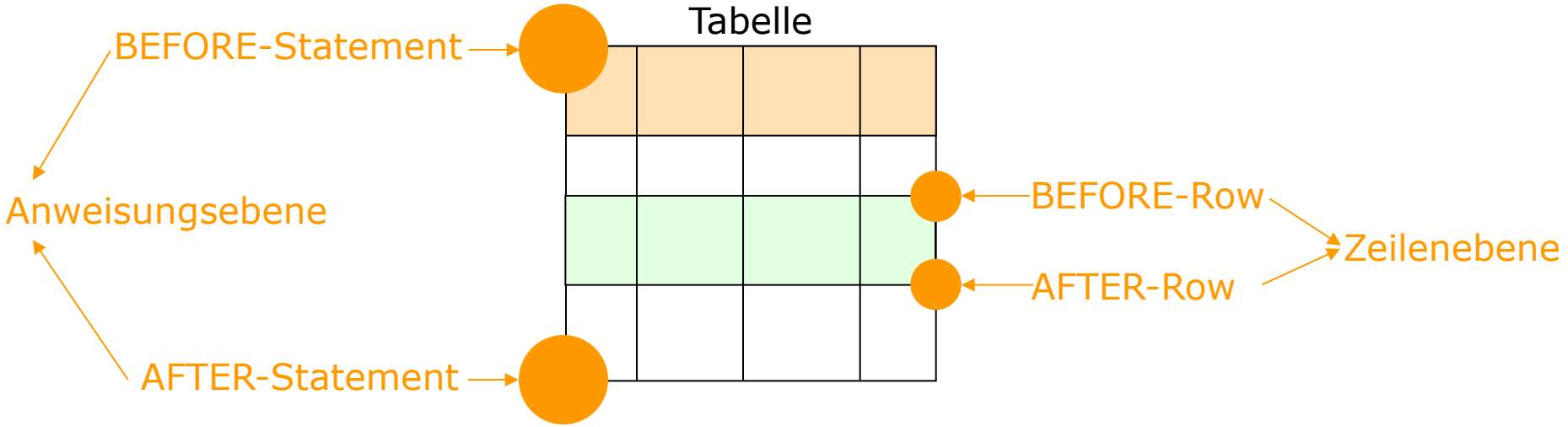
Aktives Verhalten 2/2

- Bisher
  - Integritätsbedingungen beschreiben, was innerhalb des DBS gültig und zulässig ist.
- Neue Idee
  - Spezifikation und Durchführung von Aktionen auf bestimmte Situationen oder Ereignisse im DBS
    - **Wenn** <Bedingung verletzt>
      dann <Führe Korrekturmassnahmen durch>
  - „Zusammenhangsregel" (kausale, logische oder „beliebige" Verknüpfung) statt statischem Prädikat
    - **Wenn** Mitarbeiter.Gehalt um mehr als 10% erhöht wird,
      dann benachrichtige Top-Level-Manager
  - Je mehr Integritätsbedingungen des modellierten Systems explizit repräsentiert sind, umso mehr kann das DBS „aktiv" werden!
- Oft synonyme Nutzung der Begriffe
  - Regel, Aktive Regel, ECMA-Regel, Alerter, **Trigger**

Prozedurale Integritätsbedingungen

Was ist ein Trigger?

- Trigger sind benutzerdefinierte Prozeduren, die
  - automatisch ausgeführt werden,
  - sobald bestimmte Ereignisse eintreffen.
- Trigger können einer Tabelle, einer Sicht, einem Schema oder einer Datenbank zugeordnet sein.



Prozedurale Integritätsbedingungen

Wann wird ein Trigger ausgelöst?

- DML-Ereignis: `INSERT, UPDATE, DELETE`

Notation

```
CREATE [OR REPLACE] TRIGGER <TriggerName>
    (BEFORE | AFTER | INSTEAD OF)
    (INSERT | UPDATE [OF <Spalte>]| DELETE) ON <Tabelle/View>
    [<REFERENCING-Klausel>]
    [FOR EACH ROW]
    [WHEN <Bedingung>]
    <Trigger-Aktion>
END [<TriggerName>];
```

- System- oder Benutzerereignis

Notation

```
CREATE [OR REPLACE] TRIGGER <TriggerName>
    (BEFORE | AFTER)
    (<Systemereignis> | <Benutzerereignis>)
    ON (DATABASE | SCHEMA)
    [WHEN <Bedingung>]
    <Trigger-Aktion>
END [<TriggerName>];
```

Systemereignis: `STARTUP, LOGON, SHUTDOWN, SERVERERROR,...`
Benutzerereignis: `CREATE, ALTER, DROP, GRANT, TRUNCATE, ...`

Prozedurale Integritätsbedingungen

Wofür kann ein Trigger verwendet werden?

- Automatischen **Ableiten von Spalten-Werten**
- Erzwingen von **Integritätsbedingungen** ("Implementierung" von statischen/dynamischen Integritäts- u. Konsistenzregeln)
- **Auditing** von Datenbankaktionen
- Transparentes **Ereignis-Logging**
- **Statistiken** über Tabellen-Zugriff
- **Propagierung** von Datenbankänderungen
- Synchronisieren von **Tabellen-Replikaten**
- Komplexe **Geschäftsregeln**
- **Historisierung** und **Archivierung** von Daten
- **Zugriffsschutz**
- ...

© 2026

Prozedurale Integritätsbedingungen

Trigger - Komponenten

PENDING_ORDER

part_no	reorder_quantity	o_date

INVENTORY

part_no	parts_on_hand	reorder_point	reorder_quantity

```
CREATE OR REPLACE TRIGGER Reorder_Trigger
```

Aktivierungszeit, auslösendes **Ereignis** und mit Trigger **verbundene Tabelle/Sicht**

```
AFTER UPDATE OF parts_on_hand ON INVENTORY
```

Häufigkeit (pro Tupel oder pro Datenbankoperation)

```
FOR EACH ROW
```

Bedingung (Trigger-Ausführung vom Datenbankzustand abhängig)

```
WHEN (NEW.parts_on_hand < NEW.reorder_point)
```

Aktion (Trigger-Rumpf)

```
DECLARE
  cnt NUMBER;                    /*a variable for counting                    */
BEGIN
  SELECT COUNT(*) INTO cnt       /*query to find out, if part has already     */
  FROM PENDING_ORDER             /*been reordered-if yes, cnt=1, if no, cnt=0 */
  WHERE part_no = :NEW.part_no;
  IF cnt = 0
  THEN                           /* part has not been reordered yet -> reorder */
    INSERT INTO PENDING_ORDER
    VALUES (:NEW.part_no, :NEW.reorder_quantity, SYSDATE);
  END IF;
END;
```

Prozedurale Integritätsbedingungen

Definition von Triggern

- **Trigger-Namen** müssen in Bezug auf andere Trigger **eindeutig** sein (nicht bezüglich anderer Schema-Objekte)
- Ein Trigger kann **nicht explizit verändert** werden
  - Neudefinition über `OR REPLACE`-Option im `CREATE TRIGGER`–Befehl oder
  - Löschen mit `DROP TRIGGER`– und erzeugen mit `CREATE TRIGGER`-Befehl
- Trigger können **aktiviert** (`ENABLE` = Standard) und **deaktiviert** (`DISABLE`) werden

```
ALTER TRIGGER Reorder_Trigger DISABLE;
```

```
ALTER TABLE INVENTORY DISABLE ALL TRIGGERS;
```

Prozedurale Integritätsbedingungen

Trigger-Typen: Aktivierungszeitpunkt

- **BEFORE** u. **AFTER** spezifiziert, wann (vor oder nach feuernder Anweisung) der Trigger ausgeführt werden soll

Ereignis	Häufigkeit	Aktivierungszeitpunkt: BEFORE	Aktivierungszeitpunkt: AFTER
UPDATE	ROW	✓	✓
	STATEMENT	✓	✓
DELETE	ROW	✓	✓
	STATEMENT	✓	✓
INSERT	ROW	✓	✓
	STATEMENT	✓	✓

- Anwendung von **BEFORE**-Triggern:
  - Unnötigen **Verarbeitungsaufwand** verhindern, da Trigger-Aktion feststellt, ob feuernde Anweisung weiter ausgeführt wird - ROLLBACK bei Ausnahme/Exception
  - Ableiten von **Spalten-Werten** vor Ausführung der feuernden Anweisung

Prozedurale Integritätsbedingungen

Trigger-Ereignis: DML-Anweisung

- Spezifiziert SQL-Statement-Typ, der den Trigger feuert:
  `INSERT, UPDATE, DELETE`
  - **disjunktive** Verknüpfung möglich (`OR`)
  - in diesem Fall können die **konditionale Prädikate** `INSERTING`, `UPDATING`, `DELETING` innerhalb der Aktion den Ereignis-Typ herausfinden u. in Abhängigkeit davon passenden Code ausführen
    `IF INSERTING THEN ...`
  - bei **`UPDATE`**-Ereignis kann eine **Attributliste** angegeben werden
    `IF UPDATING('salary') THEN ...`
- … und die damit verbundene **Tabelle** (genau eine)
  - auf **Views ausschließlich** `INSTEAD OF`–Trigger definierbar
  - Trigger, die auf Basis-Tabellen einer View definiert sind, werden jedoch auch **bei Operationen auf die View gefeuert**

Prozedurale Integritätsbedingungen

Trigger-Typen: Häufigkeit

- Die Häufigkeit spezifiziert, **wie oft die Trigger-Bedingung/Aktion ausgeführt** werden soll
  - **Statement-Trigger** (Default-Trigger-Typ)
    - **1 mal pro** feuerndem **Ereignis**, egal wie viele Zeilen bearbeitet werden
    - werden selbst dann ausgelöst, wenn keine Zeilen betroffen sind
    - z.B. Zugriffsschutz
  - **Row-Trigger** (Klausel: `FOR EACH ROW`)
    - **1 mal pro** vom feuernden Ereignis betroffener **Zeile**
    - werden nicht ausgeführt, wenn keine Zeilen vom ausführenden Ereignis betroffen sind
    - sinnvoll, wenn Trigger-Aktion von Daten die das feuernde Ereignis bei der Verarbeitung der einzelnen Zeilen zur Verfügung stellt, abhängt z.B. Auditing

© 2026

Prozedurale Integritätsbedingungen

Trigger: Bedingung & Aktion

- Bedingung `WHEN (NEW.empno > 0)`
  - nur bei Row-Triggern möglich
  - optionale Spezifikation eines logischen Ausdrucks (`WHEN`-Klausel), der wahr sein muss, damit Trigger feuert
  - meist verwendet für Test, ob ein Spalten-Wert < als anderer Spalten-Wert
  - erlaubte Ausdrücke: SQL-Funktionen mit booleschem Ergebnis
  - nicht erlaubt: PL/SQL u. Subqueries
- Aktion
  - **erlaubte Ausdrücke**
    - SQL
    - PL/SQL-Befehle
    - Aufruf gespeicherter Prozeduren/Funktionen
  - **nicht erlaubt**
    - DDL-Statements
    - Statements zur Transaktionskontrolle (COMMIT, ROLLBACK)

Prozedurale Integritätsbedingungen

Trigger: Zugriff auf alte/neue Spaltenwerte

- Nur bei Row-Triggern
- Anweisungen in Bedingung und Aktion haben über zwei Korrelationsnamen `OLD` und `NEW` Zugriff auf alte und neue Spalten-Werte der gerade bearbeiteten Zeile

```
IF :NEW.sal > 1000 ...
IF :NEW.sal < :OLD.sal ...
```

- Ereignisart bestimmt Sinnhaftigkeit der Korrelationsnamen:

Ereignisart	Korrelationsnamen: OLD	Korrelationsnamen: NEW
INSERT	✖	✓
UPDATE	✓	✓
DELETE	✓	✖

- Syntax in Bedingung: `NEW.column, OLD.column`
- Syntax im Aktionsteil: `:NEW.column, :OLD.column`
- `REFERENCING`-Klausel: Vergabe von anderen Bezeichnungen für `NEW` u. `OLD`: `REFERENCING NEW AS neuerStand`

Prozedurale Integritätsbedingungen

Trigger – Beispiel

Row Trigger

```
CREATE OR REPLACE TRIGGER dummy
BEFORE INSERT OR UPDATE OF salary, first_name ON employees
FOR EACH ROW
DECLARE
  ... /* variables, constants, cursors, etc. */
BEGIN
  /* PL/SQL block */
  IF UPDATING('SALARY') THEN ... END IF; ...
EXCEPTION
  ... /* exception handling */
END;
```

Auslösendes DML-Ereignis

```
INSERT INTO employees VALUES(...);
UPDATE employees SET salary = salary + 100;
```

Prozedurale Integritätsbedingungen

Trigger – Beispiel

Systemtrigger

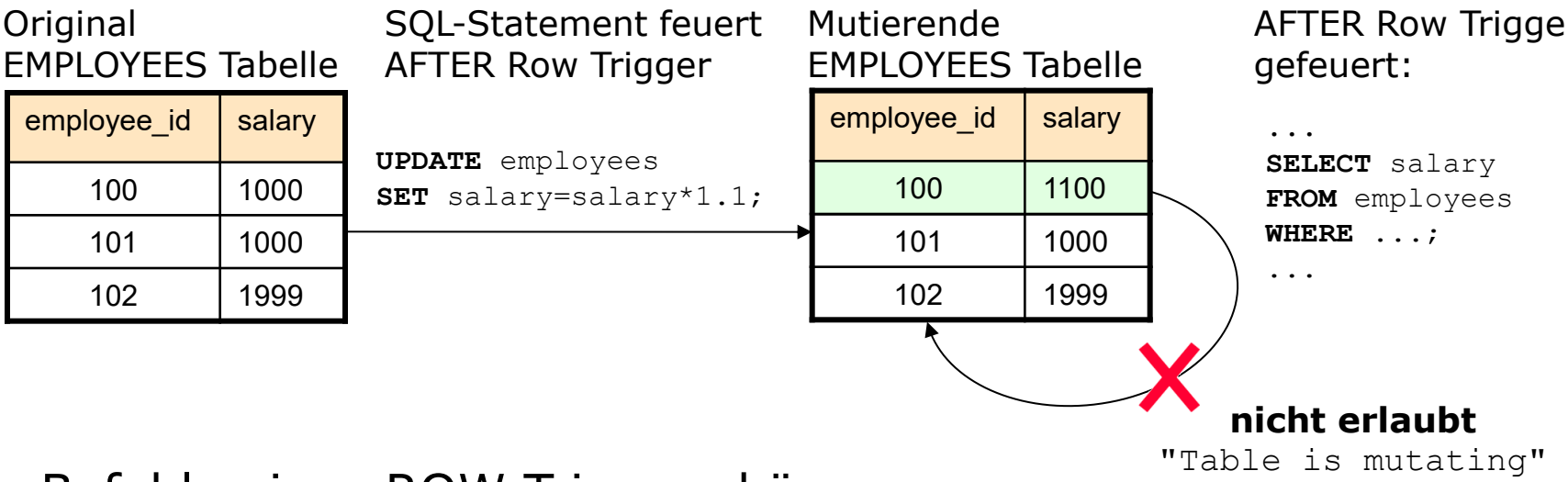
```
CREATE OR REPLACE TRIGGER logon_trigger
AFTER LOGON ON SCHEMA                /* or DATABASE */
BEGIN
 INSERT INTO log_trigger_table(user_id,log_date,action)
 VALUES (USER, SYSDATE, 'Logging on');
END;
```

auslösendes Datenbankereignis (Oracle-spezifisch)	**Mögliche Werte**
AFTER SERVERERROR	Weist den Oracle-Server an, den Trigger auszulösen, wenn eine Server-Fehlermeldung protokolliert wird
AFTER LOGON	Weist den Oracle-Server an, den Trigger auszulösen, wenn sich ein Benutzer bei der Datenbank anmeldet
BEFORE LOGOFF	Weist den Oracle-Server an, den Trigger auszulösen, wenn sich ein Benutzer von der Datenbank abmeldet
AFTER STARTUP	Weist den Oracle-Server an, den Trigger auszulösen, wenn die Datenbank geöffnet wird
BEFORE SHUTDOWN	Weist den Oracle-Server an, den Trigger auszulösen, wenn die Datenbank heruntergefahren wird

Prozedurale Integritätsbedingungen

Trigger – Mutierende Tabellen

- **Mutierende Tabelle** = Tabelle, die gerade verändert wird



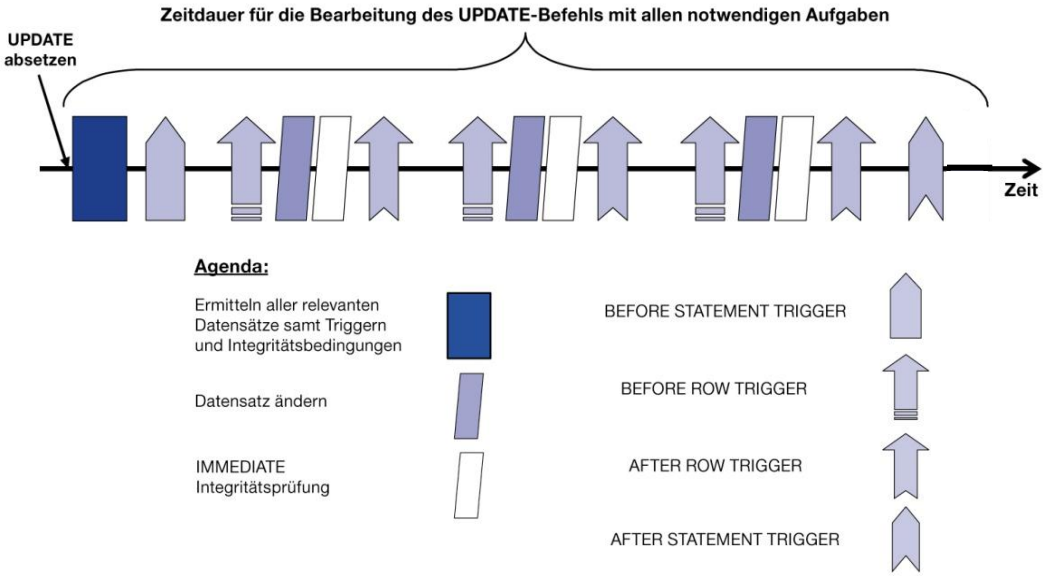
- Befehle eines ROW-Triggers können
  - mutierende Tabelle weder lesen noch schreiben

Prozedurale Integritätsbedingungen

Ausführungsmodell (in Oracle) – Multiple Trigger

- Ausführungsmodell zur Bestimmung der **Ausführungssequenz**:

1. Execute **BEFORE STATEMENT** trigger.
2. **Loop** for each row affected by SQL stateme
   a. Execute **BEFORE ROW** trigger
   b. **Lock** and **change** row, and perform **integrity** constraint checking.
   c. Execute **AFTER ROW** trigger.
3. Complete **deferred integrity** constraint checking.
4. Execute **AFTER STATEMENT** trigger.



- **Alle** aufgrund eines SQL-Statements durchgeführten Aktionen und Integritätsüberprüfungen müssen **erfolgreich sein**
- **Tritt eine Exception** auf und wird sie nicht explizit behandelt, werden alle **Aktionen** und die **triggernde Anweisung** selbst **zurückgenommen** bzw. nicht ausgeführt

Prozedurale Integritätsbedingungen

Ausführungsmodell – Kaskadierende Trigger

```
UPDATE T1 SET ...;
```

UPDATE_T1 Trigger;

```
BEFORE UPDATE ON T1
FOR EACH ROW
BEGIN
  ....
  INSERT INTO T2 VALUES(...);
  ....
END;
```

INSERT_T2 Trigger;

```
BEFORE INSERT ON T2
FOR EACH ROW
BEGIN
  ....
  INSERT INTO ... VALUES(...);
  ....
END;
```

Prozedurale Integritätsbedingungen

Mutierende Tabelle bei kaskadierenden Trigger

Kaskadierende Trigger

```
CREATE TABLE A(a NUMBER);
CREATE TABLE B(b NUMBER);

CREATE OR REPLACE TRIGGER Trg_A
AFTER INSERT ON A
FOR EACH ROW
BEGIN
    INSERT INTO B VALUES(1);
END;

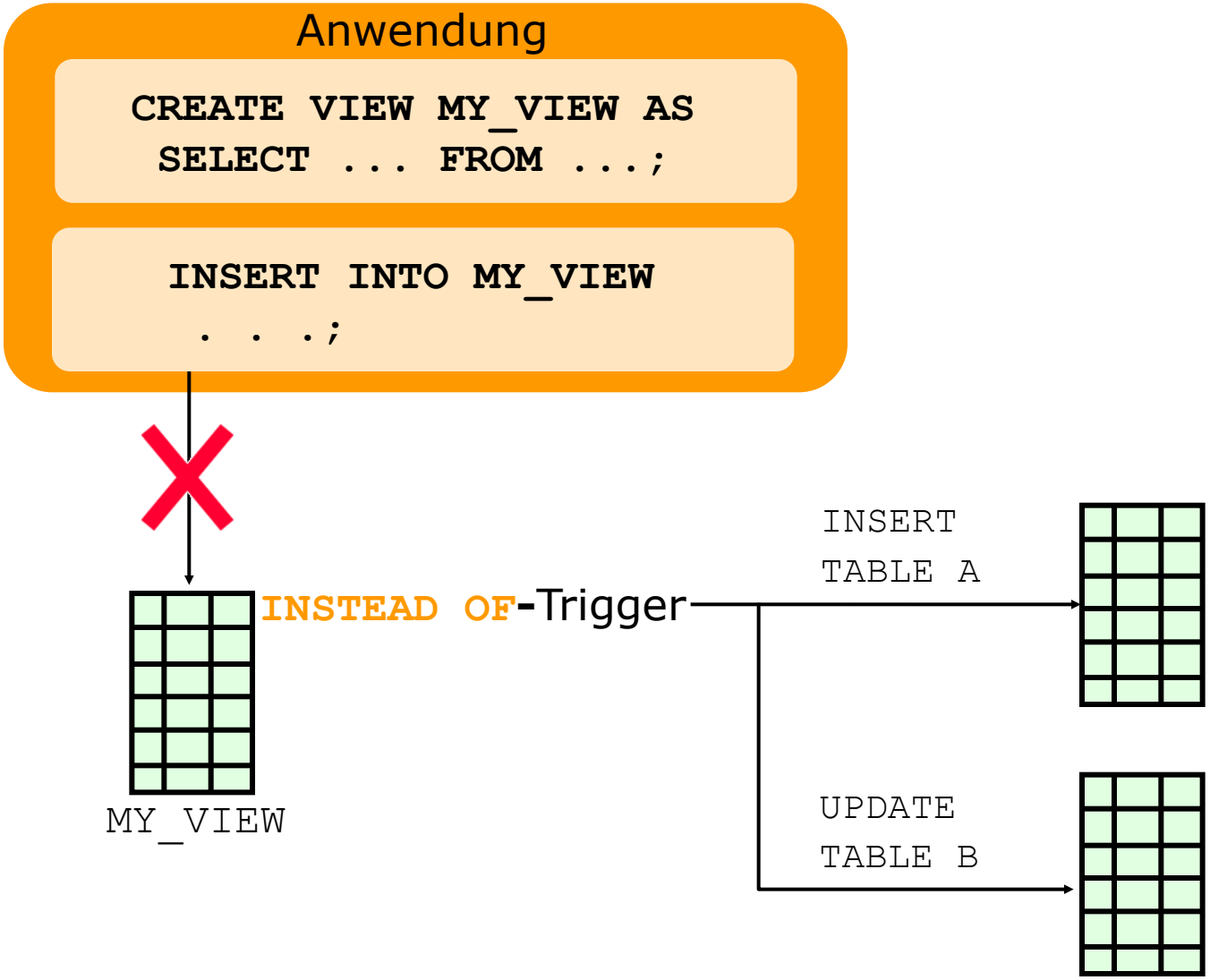
CREATE OR REPLACE TRIGGER Trg_B -- kaskadierender Trigger
BEFORE INSERT ON B
BEGIN
    DELETE FROM A;
END;
```

Auslösendes DML-Ereignis

```
INSERT INTO A VALUES(4711);
Oracle-Fehlermeldung:
ORA-04091: table A is mutating, trigger/function may not see it
```

Prozedurale Integritätsbedingungen

INSTEAD OF-Trigger – Trigger auf Sichten 1/2



Prozedurale Integritätsbedingungen

INSTEAD OF-Trigger – Trigger auf Sichten 2/2

Notation

```
CREATE [OR REPLACE] TRIGGER <TriggerName>
    INSTEAD OF <Ereignis>
    [<REFERENCING-Klausel>]
    FOR EACH ROW
    DECLARE
    <PL/SQL-Block>
END [<TriggerName>];
```

- **INSTEAD OF**-Trigger
  - leiten Änderungsoperationen um
    - anstatt die SQL-Operation (INSERT, UPDATE, DELETE, MERGE) auf der Sicht auszuführen, wird bei Erfüllung der Bedingung der Trigger-Rumpf ausgeführt
    - Trigger-Rumpf ändert die der Sicht zugrunde liegenden Basistabellen
  - arbeiten datensatzorientiert (= Row-Trigger)
  - sind nicht im Standard definiert

Prozedurale Integritätsbedingungen

Wofür kann ein `INSTEAD OF`-Trigger verwendet werden?

- Sichten für den Zugriffsschutz.
- Ergänzung von `NOT NULL`-Spalten, die in der Sicht fehlen.
- Einschränkung der Datenmanipulation, wenn z.B. bestimmte Benutzer bestimmte Werte nur innerhalb vorgegebener Grenzen verändern dürfen.
- Transparenz von Schemaänderungen, d.h. eine Änderung in der Tabellenstruktur kann mit einer Sicht und darauf definierten `INSTEAD OF`-Trigger verborgen bzw. gekapselt werden.

© 2026

Prozedurale Integritätsbedingungen

Trigger versus gespeicherte Prozeduren

Trigger	Prozeduren
Werden mit `CREATE TRIGGER` definiert	Werden mit `CREATE PROCEDURE` definiert
Data Dictionary enthält Quell-Code in `USER_TRIGGERS`.	Data Dictionary enthält Quell-Code in `USER_SOURCE`.
Werden implizit von DML aufgerufen	Werden explizit aufgerufen
`COMMIT`, `SAVEPOINT` und `ROLLBACK` sind im Trigger-Rumpf nicht zulässig.	`COMMIT`, `SAVEPOINT` und `ROLLBACK` sind im Prozedur-Rumpf zulässig.

Prozedurale Integritätsbedingungen

Trigger - Vorteile

- Verbesserte Datensicherheit
  - Detaillierte Sicherheitsprüfungen
  - Umfangreiches Auditing
- Verbesserte Datenintegrität
  - Durchsetzung dynamischer Integritätsbedingungen
  - Durchsetzung komplexer referenzieller Integritätsbedingungen
  - Gewährleistung der impliziten gemeinsamen Ausführung zusammengehöriger Vorgänge

© 2026

Prozedurale Integritätsbedingungen

Trigger – Vergleich in kommerziellen DBS

- Syntax und Semantik von Trigger in kommerziellen DBS sind sehr unterschiedlich
  - Grund: Trigger wurden erst mit SQL:1999 standardisiert, obwohl Trigger seit über zehn Jahren von kommerziellen DBS angeboten werden

	SQL:2003	Oracle	DB2	Informix	Postgres
Ereignisse pro Trigger	1	*	1	1	*
Trigger pro Ereignis	*	*	*	1	1
Granularität (ROW/STATEMENT)	R/S	R/S	R/S	R/S	R/S
Aktivierungszeit (BEFORE/AFTER)	B/A	B/A	B/A	B/A	B/A
Aktivierungsbedingung	✓	✓	✓	✓	—

Prozedurale Integritätsbedingungen

Trigger – Zusammenfassung

- Trigger sind geeignet für
  - allgemeine Aktionen, die für mehrere Anwendungen gelten
  - zentralisierte globale Operationen
- Lange Trigger-Logik sollte in Prozeduren ausgelagert werden und vom Trigger aufgerufen werden.
- Eine übermäßige Verwendung von Triggern kann zu komplexen gegenseitigen Abhängigkeiten führen, die in großen Anwendungen schwer zu warten sind.
- Trigger sollen die referenzielle Integrität unterstützen nicht ersetzen.
- Trigger können Performance- und Terminierungs-Probleme hervorrufen.
- Trigger sollten mit Bedacht eingesetzt werden (Rollback, Testfälle)

Review-Fragen

- Integrität vs. Konsistenz
- Arten von Integritätsbedingungen?
  - Beispiele
- Fremdschlüssel-Constraints?
  - `RESTRICT` vs. `NO ACTION`?
- „Verschieben“ des Prüfzeitpunktes?
- Trigger?
  - Ausführungszeitpunkte?
  - Erlaubte Konstrukte?
    - In Bedingung
    - In Aktion
  - Mutating Table?
  - `INSTEAD OF`?